

Phase 14: Response Schema

Hospital Patient Helpdesk Chatbot

Python module: 07_backend/14_response_schema.py **Jupyter notebook:**

13_notebooks/14_response_schema.ipynb **Purpose:** Define and validate the chatbot response format with answer, sources, confidence, and safety flag.

1. Phase objective

Phase 14 formalizes the response contract returned by the backend /chat endpoint. It validates real Phase 12 API responses and deliberately malformed adversarial examples so downstream UI, logging, evaluation, and monitoring code can rely on a stable schema.

The module is framework-independent and can run without Pydantic or FastAPI. When Pydantic is installed, it can create response models suitable for FastAPI/OpenAPI.

2. Folder placement

```
hospital_patient_helpdesk_chatbot/
|-- 01_data/
|   |-- processed/
|       |-- 12_api_responses.json
|       |-- 14_response_validation_results.json
|       |-- 14_response_schema_report.json
|       |-- 14_response_schema_audit.csv
|       |-- 14_failed_response_schema_checks.json
|       |-- plots/
|           |-- 14_answer_length_distribution.png
|           |-- 14_response_validation_outcomes.png
|-- 07_backend/
|   |-- 14_response_schema.py
|   |-- 14_response_schema.md
|   |-- 14_response_schema.pdf
|-- 12_tests/
|   |-- test_response_schema.py
|-- 13_notebooks/
|   |-- 14_response_schema.ipynb
-- README.md
```

All generated Phase 14 artifacts begin with 14_.

3. Response fields

Field	Rule	Purpose
request_id	Expected format like REQ-ABCDEF123456	Correlates one API response.
answer	Required, non-empty, max 2500 characters	Patient-facing answer text.
mode	Known response mode	Indicates grounded, emergency, unsafe-advice, insufficient-context, or security-refusal behavior.
citations	List of labels such as [S1]	Labels actually used in the answer.
sources	Up to 6 source records	Approved provenance supplied with the response.
retrieval_confidence	none, low, medium, or high	Retrieval quality signal, not medical certainty.
safety_flag	Boolean	True when guardrails changed or blocked output.
guardrail_action	pass, override, block, or redact	Final Phase 11 action.
risk_level	low, medium, high, or critical	Guardrail policy severity.

Field	Rule	Purpose
triggered_rules	List of rule IDs	Explains safety action when flagged.
provider and model	Required text	Traceability for generation backend.
latency_ms	Non-negative number	End-to-end API service latency.
timestamp_utc	ISO-like timestamp	Response creation time.

4. Source fields

Each source includes:

- citation
- source_file
- optional chunk_id
- optional source_type
- optional department
- optional content_category
- optional page_reference
- optional numeric score

Every citation in citations must appear in the response's own sources list.

5. Input file

01_data/processed/12_api_responses.json

This file contains real guarded API response payloads generated by Phase 12. Phase 14 appends six adversarial malformed responses during validation:

- empty answer;
- unsupported citation;
- grounded answer with sources but no citations;
- inconsistent guardrail action and safety flag;
- negative latency; and
- unknown retrieval confidence.

6. Output files

Numbered artifact	Description
14_response_validation_results.json	Full validation output with normalized valid responses and errors for invalid responses.
14_response_schema_report.json	Totals, configuration, error counts, dependency availability, and answer-length statistics.
14_response_schema_audit.csv	Compact rows without storing full answer text.
14_failed_response_schema_checks.json	Invalid response numbers and validation messages.
14_answer_length_distribution.png	Histogram of answer lengths.
14_response_validation_outcomes.png	Valid versus invalid response counts.

7. Python module code sections

Constants and contracts

The module defines accepted modes, confidence labels, guardrail actions, risk levels, answer length limit, source limit, request-ID pattern, citation pattern, and timestamp pattern.

`ResponseValidationConfig`, `ResponseSource`, `ChatResponse`, `ResponseValidationResult`, and `ResponseSchemaRunResult` provide typed, reusable boundaries.

Source validation

`validate_source()` checks citation format, required `source_file`, numeric non-negative score, and optional metadata normalization.

Response validation

`validate_response_payload()` verifies required fields, answer length, recognized mode, confidence, guardrail action, risk level, provider, model, latency, timestamp shape, citation list, source list, triggered-rule list, citation-to-source integrity, grounded citation requirements, and safety-flag consistency.

Optional Pydantic model

`create_pydantic_response_model()` lazily imports Pydantic and returns a nested response model for FastAPI/OpenAPI. If Pydantic is unavailable, the function raises a clear installation message while core validation still works.

Batch validation

`run_response_schema_validation()` loads real Phase 12 responses, appends adversarial malformed payloads, validates everything, writes JSON and CSV artifacts, creates plots, and returns output paths and counts.

CLI

```
python 07_backend/14_response_schema.py
```

Optional flags:

```
python 07_backend/14_response_schema.py --api-responses path/to/12_api_responses.json --output-dir path/to/processed
```

8. Notebook code sections

The notebook:

1. locates the project from workspace, project root, or notebook folder;
2. imports the shared response-schema module;
3. reports Pydantic availability;
4. validates one real API response;
5. demonstrates invalid response cases;
6. checks optional Pydantic model behavior;
7. runs full Phase 14 validation;
8. validates generated artifacts; and
9. displays both numbered plots.

9. Notebook versus Python module

Topic	14_response_schema.ipynb	14_response_schema.py
Main purpose	Interactive response-contract review.	Reusable canonical response validation.
Validation logic	Imports module functions.	Owens dataclasses, source checks, citation checks, safety consistency, artifacts, plots, and CLI.
Inputs	Uses Phase 12 API responses and inline adversarial examples.	Accepts any compatible API response JSON file.
Outputs	Displays examples, report, and plots.	Writes 14_ JSON, CSV, and PNG artifacts.
Dependency behavior	Reports whether Pydantic is installed.	Works without Pydantic; creates model only when installed.

The notebook does not duplicate validation rules, so both files remain aligned.

10. Automated tests

12_tests/test_response_schema.py covers:

- valid grounded response acceptance;
- empty answer rejection;
- unsupported citation rejection;
- missing grounded citation rejection;
- negative latency rejection;
- unknown confidence rejection;
- safety-flag consistency; and
- optional Pydantic model creation when installed.

In the current interpreter, pytest and Pydantic are not installed, so direct module and notebook validation were used.

11. Validation results

The included run produced:

- 18 input responses;
- 12 real API responses;
- 6 adversarial malformed responses;
- 12 valid responses;
- 6 invalid responses;
- pydantic_available: false in the current environment.

12. Safety and privacy notes

- Response-schema validation does not replace Phase 11 safety guardrails.
- Retrieval confidence is not medical certainty.
- Safety/refusal responses usually should not cite retrieved sources because their text is policy-driven.
- Audit output avoids storing full answer text.
- Response validation helps downstream UI, evaluation, and monitoring code fail early when the contract changes.